garbage-cd V2

A mark-sweep garbage collector for a toy language, and a local console that makes it visible

Project guide for your own reference: what exists, how it works, why it is built this way, what was measured, and what is not proven.

What	A toy language (integers, null, new Object(), assignment, field references, gc()) with its own mark-sweep collector, plus a dashboard that runs it and stock Java 21 side by side.
Where	Repository /Users/solstice/Desktop/gc, branch v2 (31 commits, local only), tag v1-baseline = original project.
Parts	C++17 memory subsystem and runner, JDK-only Java tools, Node 22 controller, React dashboard, benchmark harness, test and builder tooling.
Tests	12 C++ test programs (Debug, Release, ASan+UBSan), 87 controller tests, 76 dashboard tests, property tests over 10,000 seeds x 8 variants.
Verdict	Everything planned was built and runs. 4 of 5 research hypotheses passed; H2 (region overview cost) failed and is reported as such.
Generated	2026-10-18 from the repository contents and committed benchmark results.

How to use this guide: Part 1 gives the big picture and architecture, Part 2 explains the collector and the V2 techniques with diagrams, Part 3 is the dashboard and workload tools, Part 4 covers contracts, security and testing, Part 5 is the measured evidence, and Part 6 is a practical cheat sheet, limitations and glossary.

Contents

1. The big picture 4

1.1 The toy language . . . 4

1.2 What V2 added, in one table . . . 4

1.3 Why two engines and never one heap . . . 4

2. Architecture 4

2.1 Components . . . 5

2.2 The two engines side by side . . . 6

2.3 Repository layout . . . 6

3. How a run works 6

3.1 HTTP API . . . 7

3.2 Run outcomes . . . 7

4. The collector, explained 7

4.1 Mark-sweep in one picture . . . 7

4.2 How the original heap works . . . 7

4.3 What the audit of V1 found . . . 8

5. The V2 collector techniques 8

5.1 Mark spanning forest ('Why is this alive?') . . . 8

5.2 Address regions with exact cross-region counts . . . 8

5.3 Batched event tracing . . . 8

5.4 Slot reuse with generation-checked handles . . . 9

5.5 Invariants that are checked . . . 9

6. The dashboard 10

6.1 Panels . . . 10

6.2 Comparison rules (enforced in web/shared/src/compare.ts) . . . 10

6.3 A short walkthrough . . . 10

7. Writing programs: editor, object builder, CLI and the test lab 10

7.1 Builder settings . . . 11

7.2 Keeping GUI and code together . . . 11

7.3 The test lab . . . 11

8. Data contracts 12

8.1 Identifiers worth knowing . . . 12

8.2 Error codes . . . 12

9. Security and failure model 12

10. Testing and verification 13

10.1 Bugs the tests found (instead of users) . . . 14

11. Benchmarks and what they show 14

11.1 Setup in brief . . . 14

11.2 Hypotheses . . . 14

11.3 How to read the results 16
12. Project history and size **17**
12.1 Feature checklist by phase 18
13. Practical cheat sheet **18**
13.1 Setup and start 18
13.2 Tests and tools 18
13.3 Things to remember 18
13.4 Troubleshooting 18
14. Limitations and what is not verified **19**
15. Glossary **19**

1. The big picture

garbage-cd began as a small course project: a tokenizer, parser and interpreter for a tiny language, with a hand-written mark-sweep garbage collector behind a narrow interface. V2 turns that into a platform for **seeing and measuring** how a collector behaves, and for placing it next to a production JVM without pretending the two are the same thing.

1.1 The toy language

```
a = new Object();
b = new Object();
a.ref = b;     // a points at b
a = null;      // a's variable no longer roots the first object
b = null;
gc();          // nothing reaches either object: both are freed
```

- Values: integers, **null**, objects. Variables are the **roots** of the heap.
- No loops, branches, functions or arithmetic, so a program is a straight list of statements.
- Because there are no loops, collector benchmarks drive the heap interface directly instead of through scripts.

1.2 What V2 added, in one table

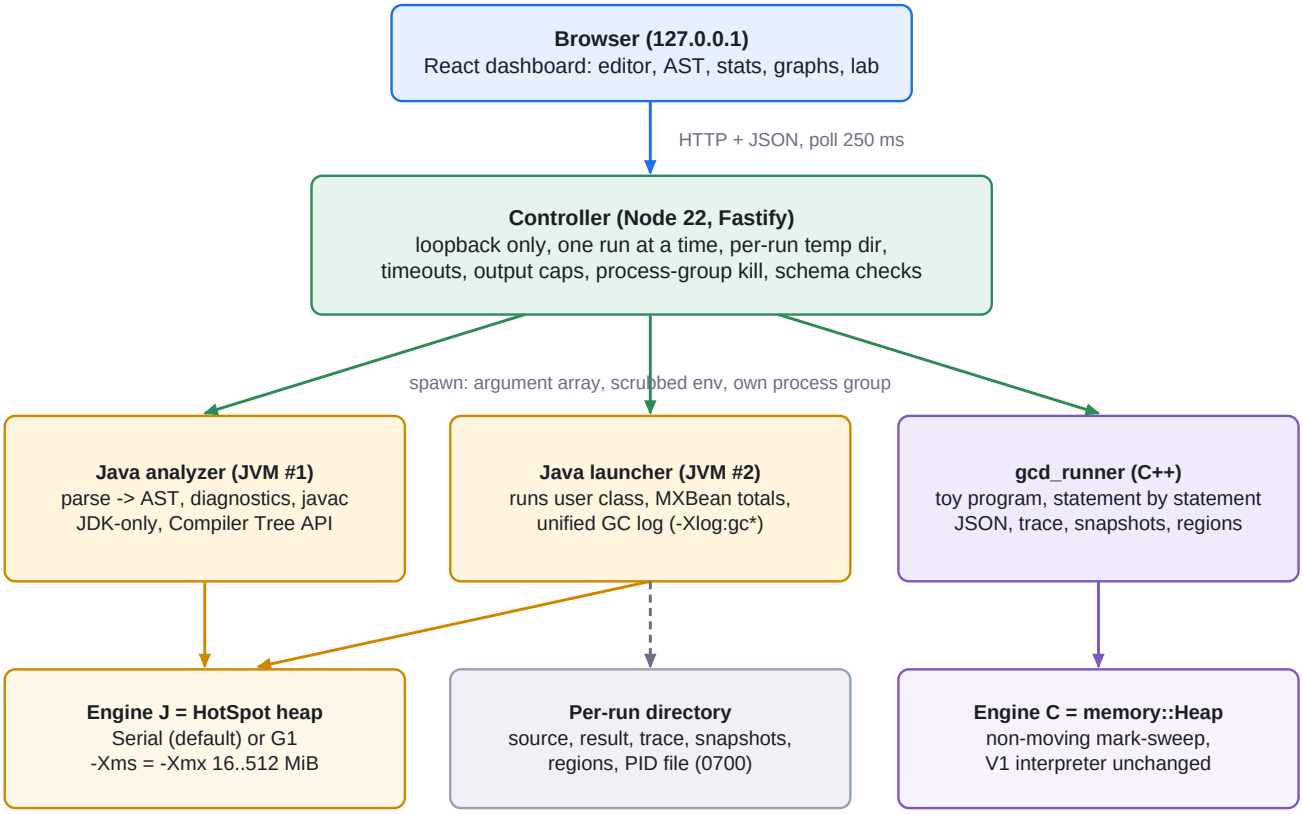
Area	What it gives you
Dashboard	Run Java or toy code in a browser; AST with exact spans; statistics with provenance badges; heap graph per collection with 'Why alive?'; event timeline; region graph; Compare; Benchmarks; Object builder; Test lab.
Controller	Local API that supervises subprocesses safely: loopback only, one run at a time, timeouts, output caps, process-group kill, schema-validated results.
Custom runtime tooling	gcd_runner executes a program statement by statement and emits structured JSON, bounded event traces, heap snapshots and region graphs; it never aborts (all errors are structured).
Collector refinements	Mark spanning forest, address regions with exact cross-region counts, batched tracing, slot reuse with generation-checked handles. None may change what is freed, and this is tested.
Evidence	Correctness matrix, property and differential tests, invariants checked after every collection, benchmarks with bootstrap confidence intervals, and a builder-driven test lab.

1.3 Why two engines and never one heap

HotSpot exposes pauses, heap sizes and bytes allocated, but not object counts, roots or the object graph. The custom runtime exposes all of that. A dashboard that put them in one table would invent numbers. So each engine has its own result type, every number is wrapped as **Metric {value, unit, provenance, source}**, missing data shows as 'Unavailable - reason' (never zero), and a Compare tab refuses deltas that mix engines or units.

Provenance badges: M = measured (clock, counter, MXBean, GC log), E = estimated (documented formula), R = reconstructed (graph layouts), U = unavailable.

2. Architecture



Two engines, never one heap: results are separate and every number carries its provenance.

Figure 1 - The whole system. The controller is the only component the browser talks to; each run starts one or two short-lived tool processes.

2.1 Components

Component	Where	Responsibility
Dashboard	web/client/src	React 18 + Vite, CodeMirror 6 editor, Cytoscape graphs, plain SVG timeline; one reducer store; polls runs every 250 ms; error boundary per panel.
Controller	web/server/src	Fastify on 127.0.0.1:4177. Routes, request validation, adapters for both engines, subprocess supervision (process.ts), Ajv validation of every payload, trace paging, benchmark listing, static serving of the built dashboard.
Shared library	web/shared/src	Types, the builder/generator and lab checks, and the comparison rules; used by the dashboard, controller tests and CLI scripts.
Java analyzer	java/analyzer	JDK Compiler Tree API: parse, find the public type and main, build the AST (capped at 5,000 nodes, exact spans, synthetic flag), compile with -proc:none -Xlint:all.
Java launcher	java/launcher	Calls main reflectively, catches Throwable, writes MXBean totals in a shutdown hook (so System.exit still reports).
Runner	tools/gcd_runner	C++17. TracingHeap decorator over memory::Heap, statement stepping, structured errors, toy AST JSON, EventBuffer, SnapshotWriter, RegionGraphWriter.
Memory subsystem	include/memory, src/memory	memory::Heap (implements gc::IHeap), mark/sweep, diagnostics, GcObserver + HeapView, RegionTable, invariant checks.
V1 interpreter	include/interpreter, src/interpreter, include/runtime, src/runtime	Unchanged lexer, parser, interpreter. Frozen so that V1 behaviour stays reproducible as benchmark variant A.
Contracts	contracts/	10 JSON Schemas (Draft 2020-12) with examples.
Bench	bench/	heap_bench harness (W1-W7), JVM workloads (J1-J5), run_bench.sh, analyze.py, committed results.

2.2 The two engines side by side

	Engine J: standard JVM	Engine C: custom runtime
Program	Java 21, one public class with main	Toy language
Heap	HotSpot: Serial (default) or G1; -Xms = -Xmx (16..512 MiB, default 64)	memory::Heap: non-moving, non-generational mark-sweep
Observable	GC count and pause (MXBeans), heap sizes and causes (unified GC log), allocated bytes	Objects, edges and slots per collection; mark/sweep time; trace; snapshots; regions
Not observable	Object counts, roots, object graph (shown as Unavailable)	Nothing in the toy heap is hidden; bytes are estimates
Time limit	Analysis 15 s; execution 10 s (selectable to 30 s)	5 s

2.3 Repository layout

```
garbage-cd/
  include/gc_interface.hpp     the only contract between interpreter and heap (UNCHANGED)
  include/memory, src/memory   memory::Heap, mark.cpp, sweep.cpp, observer, diagnostics, region_table.hpp
  include/interpreter ...      V1 interpreter + runtime (UNCHANGED)
  tools/gcd_runner/            runner, event buffer, snapshot + region writers, toy AST
  tests/                       V1 smoke tests; tests/v2/ = correctness, property, differential, region,
                               handle-reuse, diagnostics, event-buffer, snapshot, runner tests
  java/                        analyzer + launcher, build.sh -> build/java/gcd-java-tools.jar
  web/{shared,server,client}   npm workspaces; web/scripts = smoke, gen-toy, lab
  contracts/                   schemas + examples        bench/   harness, workloads, results
  scripts/                     setup.sh, start.sh        docs/v2/ architecture, contracts, methodology,
                                                           security, report, PROGRESS (full log)
```

3. How a run works

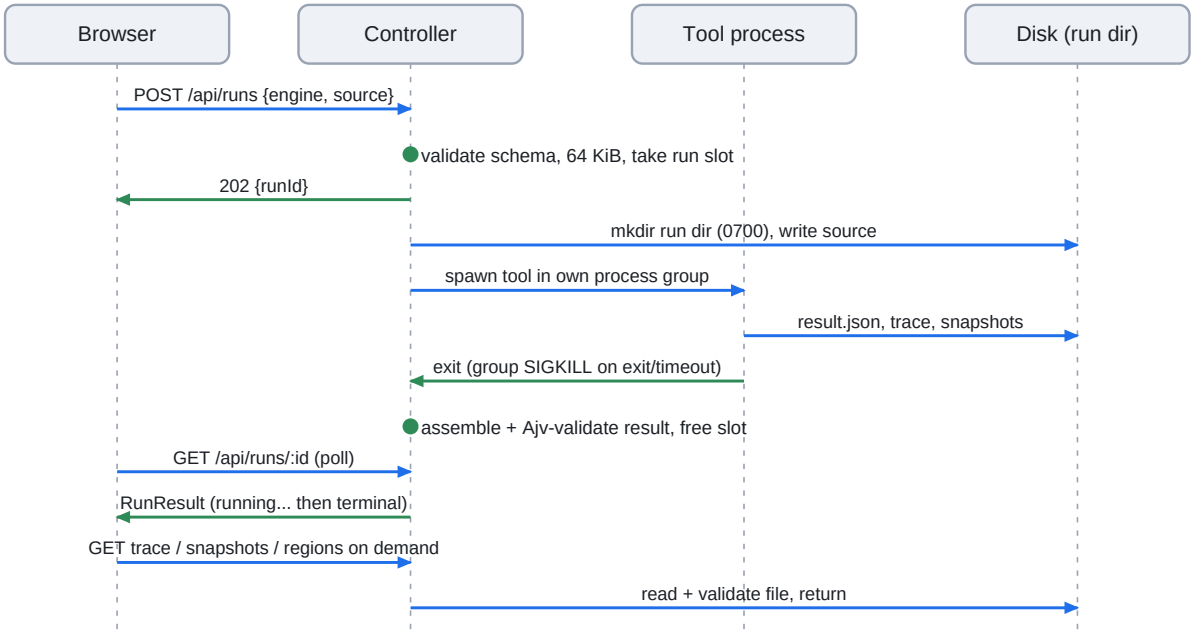


Figure 2 - Lifecycle of one run. The slot is released in the same step the terminal status becomes visible (a race fixed in phase 4).

3.1 HTTP API

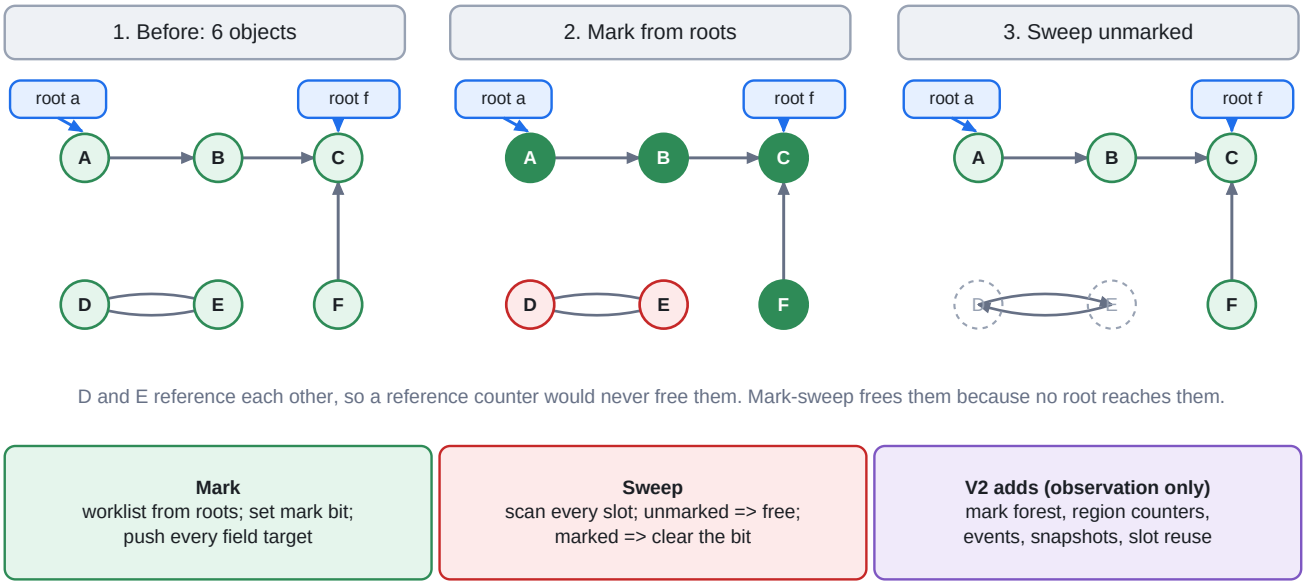
Endpoint	Purpose
GET /api/health	Tool versions, JDK found and supported (21), analyzer and runner found, platform limits.
GET /api/samples	Bundled programs: 5 Java, 4 toy.
POST /api/analyze	Parse-only analysis (Java syntax diagnostics and AST, or toy AST). No execution.
POST /api/runs	202 {runId}; 409 BUSY if a run is active; 400/413/415 for bad requests.
GET /api/runs/:id	RunResult; partial while running.
POST /api/runs/:id/cancel	Run ends as 'cancelled' within a second.
GET /api/runs/:id/trace?afterSeq=&maxEvents=	Paginated NDJSON batches (up to 50,000 events), whole batches so no sequence number is skipped or repeated.
GET /api/runs/:id/snapshots/:index	Heap snapshot (at most 2,000 objects).
GET /api/runs/:id/regions?gc=n|end	Region graph after a collection or at program end.
GET /api/benchmarks, /api/benchmarks/:id	Committed benchmark summaries (quick runs are never listed).

3.2 Run outcomes

Status	Meaning	Typical cause
succeeded	Finished normally	-
compile_error	javac or parse failure	syntax or type error, no main method
program_error	The program failed	uncaught exception, OutOfMemoryError, System.exit(n), toy runtime error
timeout / output_limit / resource_limit	A limit was hit; the whole process group is killed	infinite loop; > 1 MiB of output; CPU limit (Linux)
cancelled	Stopped by the user	Stop button
internal_error	Tool crashed or result failed validation	bug or missing tool

4. The collector, explained

4.1 Mark-sweep in one picture



D and E reference each other, so a reference counter would never free them. Mark-sweep frees them because no root reaches them.

Figure 3 - Two roots (a and f). Mark finds everything reachable (A, B, C, F). Sweep frees what was not marked, including the D-E cycle.

4.2 How the original heap works

- Objects live in a vector of slots; a **handle** is the slot index. Roots are a map from variable name to handle.
- **collect()** = mark (iterative worklist from the roots, setting a mark bit and pushing each field target) then sweep (scan every slot; free unmarked, clear marks of survivors).

- The interpreter only ever sees gc::IHeap and an opaque ObjectHandle. memory::Heap can change freely behind that interface.
- Reachability is precise: roots are named variables, not scanned stacks, so there are no false retentions.

4.3 What the audit of V1 found

ID	Finding	V2 response
D1	Slots never reused: collection cost tracks every allocation ever made	Variant E: slot reuse with generation-checked handles; measured
D2	Any lexer/parser/runtime error aborted the process (exit 134)	Runner catches and classifies; main.cpp untouched
D3	Tests were no-ops under NDEBUG	Always-on CHECK macro; -UNDEBUG on old tests; a Release canary proves checks are live
D4/D5	Tests did not check reclamation; no timings or counts	Correctness matrix; GcDiagnostics per collection
D6-D10	No source positions; stoi overflow; CWD-relative default path; forgeable handles; stale README	Statement spans, runner catches exceptions, explicit paths, generation checks, README rewritten

5. The V2 collector techniques

The original brief asked for a 'forest of regions'. Analysis showed that cannot work as stated: two regions referencing each other form a cycle, so a region graph is a general directed graph. What **is** a forest is the mark phase's first-discovery edges. Region summaries also cannot speed up marking (marking only visits reachable objects), and 'regions for safety' is a different mechanism (generation checks). So V2 builds four separately switchable pieces and measures their costs, with one rule: **none may change what is freed**.

5.1 Mark spanning forest ('Why is this alive?')

- During mark, each object remembers who first reached it: a parent object and field, or a root name. Following these links from any live object always ends at a root.
- The mark loop is a template on whether this is recorded, so an unobserved collection does no parent bookkeeping. The records live in a side table in Heap, which keeps ordinary objects at 64 bytes (a pilot benchmark caught them growing from 48 to 152 bytes).
- The dashboard's 'Why alive?' highlights the retention path from the root to the clicked object.

5.2 Address regions with exact cross-region counts

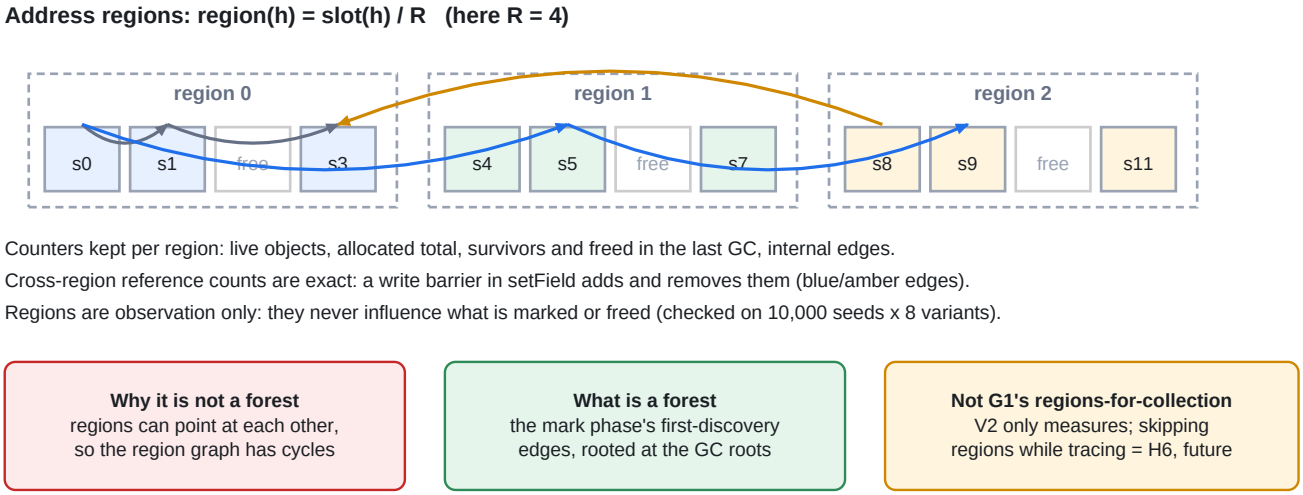


Figure 4 - Regions are blocks of R consecutive slots. A write barrier keeps exact cross-region reference counts; verifyRegions() recomputes everything from scratch.

5.3 Batched event tracing

- Events (allocate, root set/clear, field write, gc begin/end, and at level 'full' every mark visit and sweep free) are appended to a buffer of B events (1 to 8,192, default 1,024) and flushed as one NDJSON line per batch.
- Bounds: 1,000,000 events or 64 MiB. After the cap only structural events (gc begin/end) are kept and drops are counted by kind. A trailer line records completeness; a missing trailer means the runner died.

- Trace levels: off, gc, mutator, full. Replaying the mutator events into a model heap reproduces every collection's freed set (tested).

5.4 Slot reuse with generation-checked handles

Handle encoding (gc::ObjectHandle wraps one size_t; the interface is unchanged)

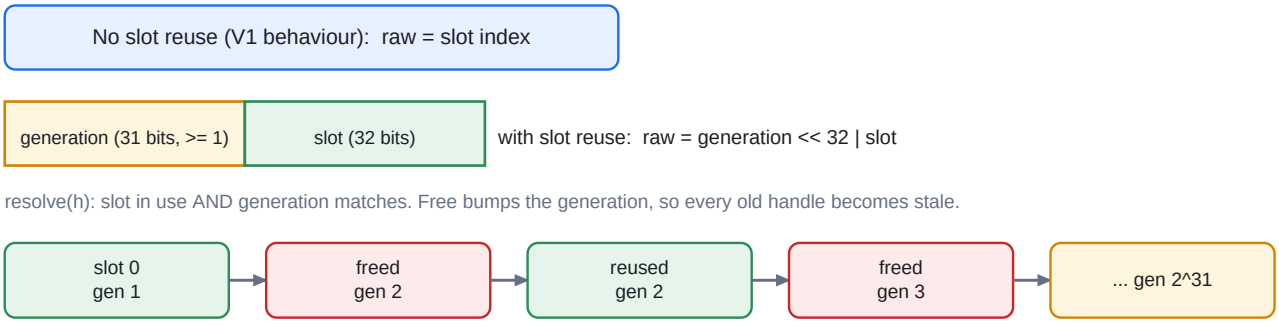


Figure 5 - Handle layout and slot life cycle. The interface type is untouched; only the heap's reading of raw() changes.

5.5 Invariants that are checked

ID	Invariant	Checked by
I1	Every root is invalid or resolves to an in-use object	verifyInvariants after every collection (Debug), tests
I2	After collect, in-use objects are exactly those reachable at the start (vs an independent BFS)	property + differential tests
I3	No mark bit remains set after a collection	verifyInvariants
I4	Every stored field target resolves (stale writes are stored as null)	verifyInvariants, hardening tests
I5	live after = allocations - total freed = count of in-use slots	verifyInvariants (valid right after a collection)
I6	Region counters equal a from-scratch recomputation	verifyRegions after every operation in tests
I7	Mark-forest edges are real field/root edges and form a forest rooted at roots	verifyInvariants
I8	With reuse, a handle from before a free never resolves again; free list is consistent	handle_reuse_tests, verifyInvariants
I9	Observer, regions, reuse on or off: identical freed sets per collection	differential_tests (10,000 seeds x 8 variants), harness digests

6. The dashboard

6.1 Panels

Panel	What you see	Notes
Toolbar	Engine, sample, Regions size, Slot reuse, Run, Stop, Reset, status pill, engine banner	Banner always says which engine and heap you are on
Editor	CodeMirror (Java mode for Engine J); gutter markers for diagnostics	Click an AST node to highlight its span; compile errors move the cursor
AST	Collapsible tree: kind, label, line:column; synthetic and truncation badges	Java from javac exact positions; toy statement spans derived from ';'
Statistics	Totals and per-collection tables with M/E/R/U badges	Unavailable shows its reason
Output	Structured errors with jump links, diagnostics, stdout, stderr	stdout capped at 64 KiB, killed at 1 MiB
Heap graph	Objects as nodes, fields as labelled edges, roots as diamonds; snapshot slider; 'Why alive?'	Green = survived, red dashed = will be freed, blue = live at program end; max 2,000 objects
Regions	One box per region sized by live objects, edges = cross-region counts	Click a region to filter the heap graph to it
Timeline	Allocations, root changes and field writes per statement; GC markers; live-object line	Click a GC marker to open its snapshot
Compare	Two runs from this session side by side	Delta only for same engine, key and unit (see 6.2)
Test lab	Run preset and random scenarios under four heap variants	Section 7
Benchmarks	Hypothesis verdicts with evidence, ratio chart with CI whiskers, H2 table, HotSpot reference rows	Reads committed results via /api/benchmarks

6.2 Comparison rules (enforced in web/shared/src/compare.ts)

- A delta appears only for the **same engine**, the **same metric key** and the **same unit**, with neither side unavailable.
- Cross-engine concepts (collections, memory allocated, memory reclaimed, peak or live size) are listed side by side with a note that their bases differ - never a delta.
- Timings from different engines are never combined into a speed-up figure.
- The sentence 'Separate runtimes, separate programs. Differences reflect language, runtime, object model and collector together.' is always shown.

6.3 A short walkthrough

- Choose Custom runtime, sample 'cycle', Run: both objects are freed; the heap graph at GC #1 shows two red dashed boxes with a 'next' edge each way.
- Choose sample 'chain', click the tail object in the heap graph: the whole path from the root variable lights up and the text says why it is alive.
- Set Regions to 16 and run a program with 20 objects: the Regions tab shows two regions with cross edges and the line 'counters verified against a full recomputation'.
- Switch to Standard JVM and run 'Allocation': Statistics shows measured GC counts and pauses and 'Unavailable - HotSpot does not expose...' for object counts.

7. Writing programs: editor, object builder, CLI and the test lab

All three ways produce ordinary toy code, so they can be mixed. The code is always the single source of truth. The language has no comments, so the builder keeps its settings outside the program; the same pure generator powers the GUI, the CLI and the tests.

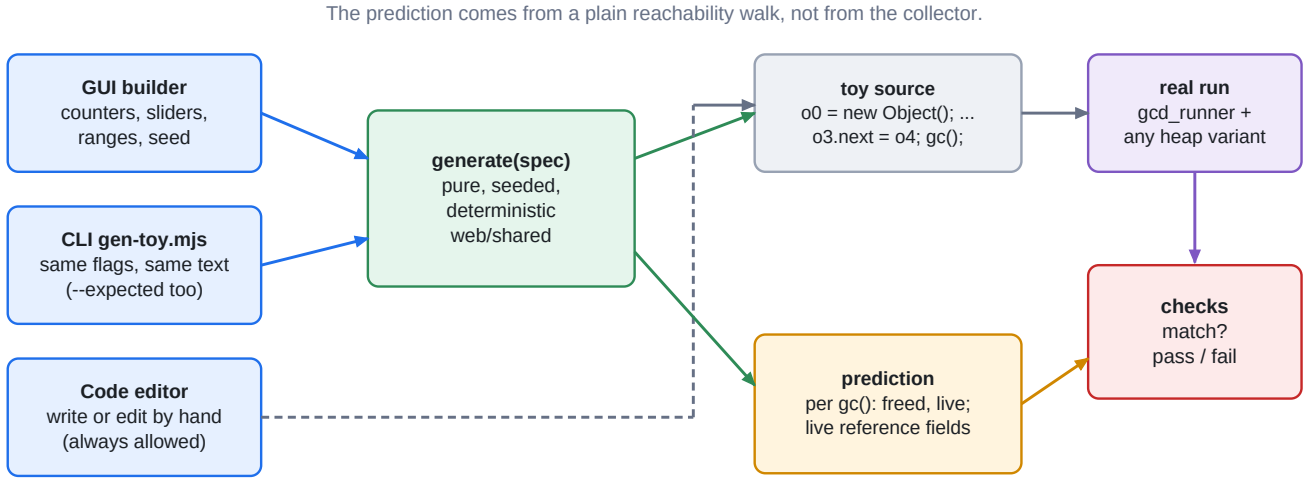


Figure 6 - One generator, three front ends. The prediction is computed by plain reachability and compared with what the real collector did.

7.1 Builder settings

Setting	Meaning
Objects	Counter (stepper, field, slider), 1 to 3,000. A program larger than 64 KiB is flagged and not run (about 1,300 objects with no extra links).
Cycles vs chains	Percentage of groups that are cycles. Objects are split into consecutive groups; a chain links next-pointers, a cycle also closes back to its head.
Chain / cycle length	Inclusive ranges per group (cycles at least 2); the last group may be shorter.
Extra links per object	A range (0 to 8) of additional references from each object to a random other object.
Locality window	0 = anywhere; otherwise extra links stay within that many objects (useful to test regions).
Kept roots %	Share of groups whose head variable stays rooted at the end.
Unroot rounds	The other groups are dropped over 1 to 8 rounds, each followed by gc(), giving progressive death.
gc() while rooted	Collect once while everything is rooted (nothing may be freed).
Seed	Same spec + seed = byte-identical program everywhere (GUI, CLI, tests).

7.2 Keeping GUI and code together

- **Generate into editor** writes the program. **Live sync** then regenerates on every change until you edit the code by hand; a banner says when the builder no longer overwrites.
- The builder predicts, for every gc(), how many objects are freed and live, plus the live reference fields. After a run it states whether the real run matched - only if the run used exactly this generated code.
- CLI: `node web/scripts/gen-toy.mjs --objects 200 --cycle-share 60 --chain-length 3-8 --extra-links 0-2 --locality 10 --keep-roots 30 --rounds 3 --seed 7 > p.gc` then `build/gcd_runner --source p.gc --result r.json`. The builder shows the matching flags for whatever you set.

7.3 The test lab

- Ten presets (single chain, single cycle, all rooted, all garbage, mixed 50/50, dense cross-links, local links, many singletons, 8-round death, 900-object heap) plus N seeded random scenarios.
- Each runs under four heap variants: plain, regions of 16, slot reuse, regions of 64 + slot reuse.
- Checks: run status, allocations, per-collection freed / live-before / live-after, totals, the program-end snapshot, and for regions the verified flag plus live, allocated and edge totals.
- Available as the Test lab tab, as `npm run lab` against a running controller, and in the server test suite.

Result: the real collector matched the prediction in every case (64/64 from the CLI, 56/56 in the dashboard, all presets and 25 random scenarios x 4 variants in the test suite). The tests can fail: a deliberately wrong prediction is caught, and the generator's own unit tests found a real bug (a locality link wrapping around).

8. Data contracts

Every payload crossing a process or HTTP boundary has a versioned JSON Schema (Draft 2020-12). The controller validates results with Ajv in strict mode; a failing payload becomes internal_error with SCHEMA_VIOLATION naming the path.

Schema	Describes
run-request	POST /api/runs body: engine, source (<= 65,536 chars), per-engine options (collector, heapMiB, timeoutMs; traceLevel, batchSize, snapshots, regionSize, slotReuse)
run-result	Envelope for both engines: status, phases, output, errors, static analysis, engine result, trace summary, limits, tool versions
jvm-engine-result / custom-engine-result	Engine totals, per-collector or per-collection rows, snapshot and region-graph indexes
static-analysis	AST (Java or toy), diagnostics, main class, package
structured-error	Code, phase, message, optional location and detail
heap-snapshot, trace-batch, region-graph	Snapshot (<= 2,000 objects), one NDJSON event batch, region counters and edges
benchmark-summary	Output of bench/analyze.py

8.1 Identifiers worth knowing

- runId: ULID. It is also the run directory name and is generated, never user-controlled.
- Object ids in snapshots and traces: the handle as a decimal string. With slot reuse it is generation << 32 | slot (large numbers - that is normal).
- Trace event: [seq, kind, statement, ...args] with kinds SB/SE, A, RS/RC, FW, GB/GE, MV, SF; names are interned per batch.
- Region id = slot / region size; slotEnd is inclusive.

8.2 Error codes

SOURCE_TOO_LARGE, BAD_REQUEST, BUSY, LEX_ERROR, PARSE_ERROR, COMPILE_ERROR, NO_MAIN_METHOD, RUNTIME_ERROR, UNCAUGHT_EXCEPTION, OUT_OF_MEMORY, TIMEOUT, OUTPUT_LIMIT, RESOURCE_LIMIT, PROGRAM_EXIT, PROCESS_CRASH, CANCELLED, ENGINE_UNAVAILABLE, SCHEMA_VIOLATION, INTERNAL.

gcd_runner exit codes: 0 ok, 2 program error (valid result written), 3 usage or I/O error, 4 internal error.

9. Security and failure model

The tool is for **one user running their own code on their own machine**. It contains accidents (loops, memory blow-ups, output floods) and blocks drive-by web pages. It is **not** a sandbox against hostile code.

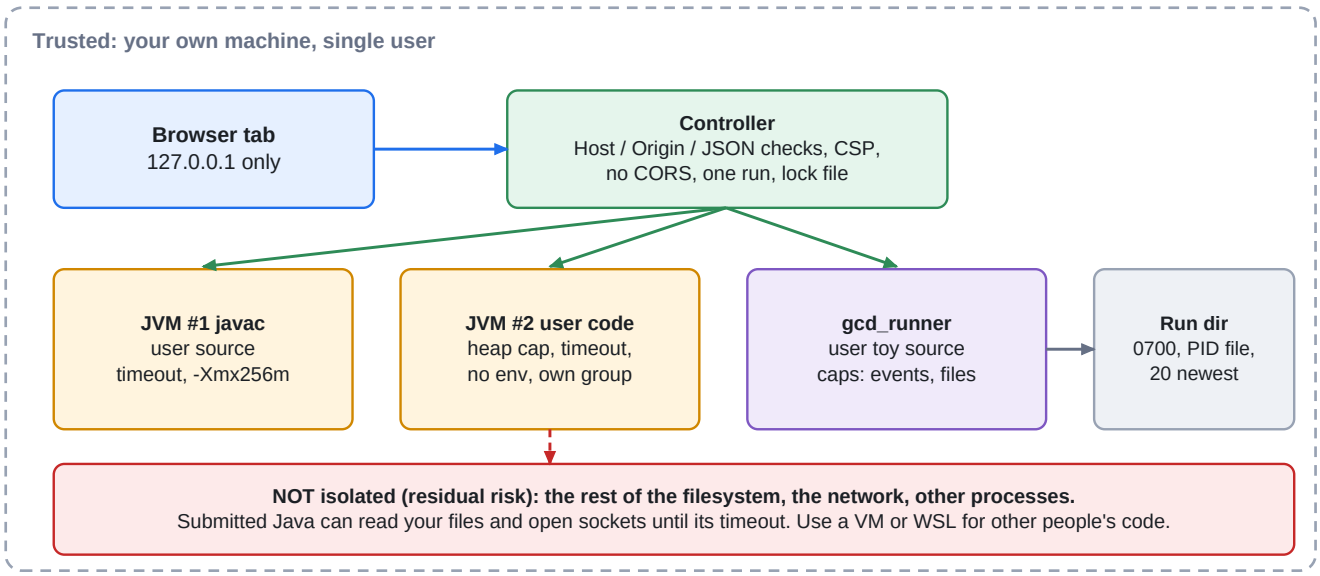


Figure 7 - Trust boundaries. The dashed line from the user's Java code to the rest of the machine is the honest summary of the residual risk.

Threat	Mitigation (and where it is tested)
Program loops, allocates, floods output, starts threads	Timeouts, heap cap + ExitOnOutOfMemoryError, output caps (kill at 1 MiB), process-group kill; hardening tests cover loops, allocation bombs, 20,000 threads, direct buffers, stack overflow, stderr floods
Program leaves children running	Group SIGKILL on every exit, timeout, cancel and shutdown; tests start and abandon sleep processes
Another website calls the API (CSRF, DNS rebinding)	Bind 127.0.0.1; Host must be localhost/127.0.0.1 on the controller's port (no port means 80, rejected); cross-origin Origin rejected; POST needs application/json so browsers must preflight, which is never approved; no Access-Control headers
Environment injection	Children get an allow-list environment; JAVA_TOOL_OPTIONS, _JAVA_OPTIONS, JDK_JAVA_OPTIONS, CLASSPATH never reach the program (tested by setting them)
Path traversal	Run ids are generated; request values are only matched against directory listings or an in-memory file map, never concatenated into paths
Leftovers; two controllers sharing a run dir	Per-port run directory (0700), startup sweep kills PID-file groups, lock file refuses a second live controller, 20-run retention, graceful shutdown on SIGINT/SIGTERM/SIGHUP
Browser-side issues	CSP default-src 'self', frame-ancestors 'none', nosniff, X-Frame-Options DENY, no-referrer, no-store on the API

Residual risks (say so in any demo): submitted Java can read files you can read, open network connections and start processes until its timeout; prlimit applies per process and only on Linux (not exercised in this repository's tests); on macOS a program can write to disk until its timeout; a child that creates its own session escapes the group kill; the controller has no authentication. Use a VM or WSL for other people's code. A bubblewrap hardened mode is documented, not built.

10. Testing and verification

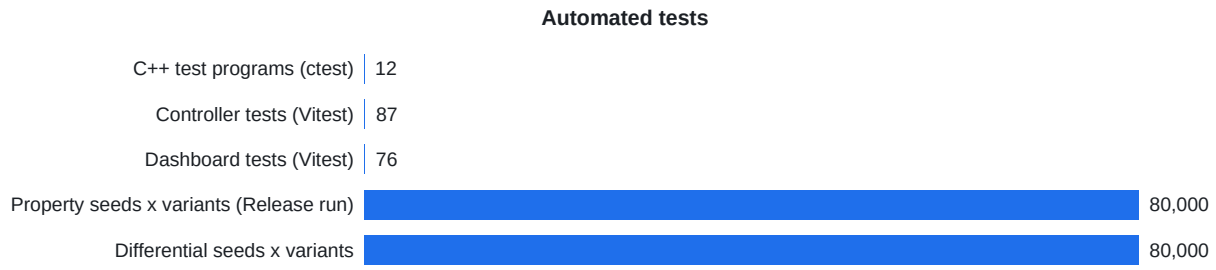


Figure 8 - Test volume. The last two bars count generated programs, not test files: 10,000 seeds x 8 variants each.

Layer	What it proves
tests/v2/gc_correctness_tests	G.1 matrix: rooted survives, garbage reclaimed, chains, shared targets, cycles (self, 2-, 5-cycles, cycles off garbage), root replacement, 100 idle collections, stats arithmetic, invalid handles
gc_property_tests	Seeded random programs checked against an independent 60-line BFS oracle, on 8 variants; verifyInvariants after every collection; region counters after every operation (first 200 seeds) or every 16th
differential_tests	Freed allocSeq sets and live counts identical across variants A', B64, B16, C, D, E, E+B, E+D (invariant I9)
region_tests / handle_reuse_tests	Counters equal recomputation after every operation (R in 1,16,64,256, reuse on/off); stale handles never resolve; LIFO free list; generation retirement at a lowered limit
diagnostics, event_buffer, snapshot tests	Mark forest is made of real edges; batch boundaries for B = 1, 3, 1,024; overflow keeps structural events; trailer; snapshot truncation keeps roots
runner_tests.sh	Sample counters match V1, all error probes give exit 2 with structured errors, limits, UTF-8 JSON escaping, flag validation, variants never change reclamation
Release canary, ASan + UBSan	Checks stay live under NDEBUG; no memory errors or UB in the whole suite
Controller tests	All G.3 rows against a real JDK 21, GC-log golden files (Serial and G1), AST goldens and span round-trips, trace paging, regions, benchmarks routes, 20 hardening tests, builder-driven lab
Dashboard tests	Provenance badges, unavailable rendering, compare rules, graph model and 'Why alive?', region and benchmark panels, builder state, lab

10.1 Bugs the tests found (instead of users)

Where	Bug	Found by
Controller	Analyzer launched with -jar although the jar has no main class: every Java run failed	G.3 suite
Controller	run-request schema rejected by Ajv strict mode: server could not start	G.3 suite
Parser	GC-log regex missed padded tags, so no pauses were parsed	Golden GC-log files
Controller	Single-run slot released after the result file write: spurious 409 BUSY	Flaky test, then reasoning
Controller	Host header without a port accepted; children left behind on normal exit; controller exit left JVMs running; two controllers swept each other's runs	Security review tests
Heap	Mark forest on every object tripled Object size (48 to 152 B)	Pilot benchmark
Generator	Locality link wrapped from last object to first	Generator unit test
Test code (x3)	Dangling HeapView in range-for; observer owner moved (found by ASan)	ASan

11. Benchmarks and what they show

11.1 Setup in brief

- Harness drives gc::IHeap directly from seeded generators: W1 churn (10^6 allocations, 1% kept), W2 linked list 10^5, W3 binary tree depth 16, W4 cycles, W5 10^6 random setField, W6 six rounds of 10^6 garbage, W7 fan-in hub.
- Variants: A = V1 (git tag, separate worktree, same compiler and flags); A' = V2 with everything off; B regions; C tracing (levels x batch sizes); D regions + tracing; E slot reuse; E+D.
- 30 samples per cell (3 fresh processes x 10 measured after 5 warm-up); variants interleaved; 95% bootstrap CIs of ratios to A'; 'real' only if the CI excludes 1 and the effect exceeds max(2%, A' vs A' noise floor).
- Apple M4, 10 cores, on battery, no CPU pinning (macOS). Raw files, tables, figures and a REPORT.md are committed under bench/results/.

11.2 Hypotheses

ID	Hypothesis	Verdict	Evidence (short)
H1	Region metadata faithfully represents cross-region references	PASS	0 mismatches: 1,000 x 8 region programs, 10,000 x 8 property programs
H5	Observation never changes collection	PASS	0 mismatches: 10,000 x 8 differential programs; 42 harness digest cells
H2	A region overview is cheaper than building it from a full snapshot	FAIL	Export 26.8x faster at 1,000 objects, 6.6x at 10,000, only 1.2x at 100,000 and 1.4x at 1,000,000 (needs >= 2x)
H3-mutator	Batching lowers telemetry overhead at trace level mutator	PASS	B=1,024 removes 83-94% of B=1's overhead on 7 of 7 workloads
H3-full	Batching lowers telemetry overhead at trace level full	PASS	Same at full level: 7 of 7 workloads
H4	Region benefits are not cancelled by their costs	PASS	Barrier about 58 ns per setField; 69 B region metadata per live object; break-even <= 1 overview per collection on 6 of 7 workloads (W5: 24)
DORMANT	Dormant V2 hooks (everything off) cost under 3 % of collect time on W6 versus V1 (I.3 acceptance)	PASS	A'/A collect ratio on W6 = 0.956 (within 3%). Whole-run A'/A is 1.06-1.15x, and 1.43x on W5

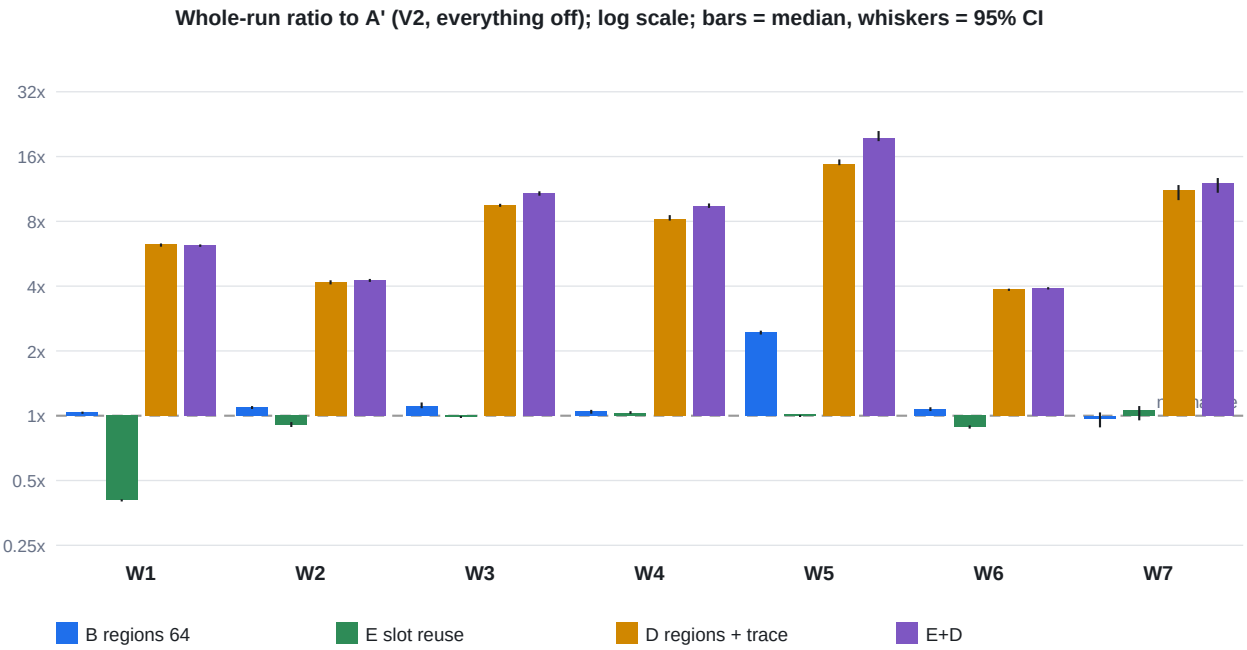


Figure 9 - Whole-run time relative to A'. Regions (B) cost almost nothing except on W5 (every operation is a write); slot reuse (E) helps churn most; full tracing (D, E+D) is expensive.

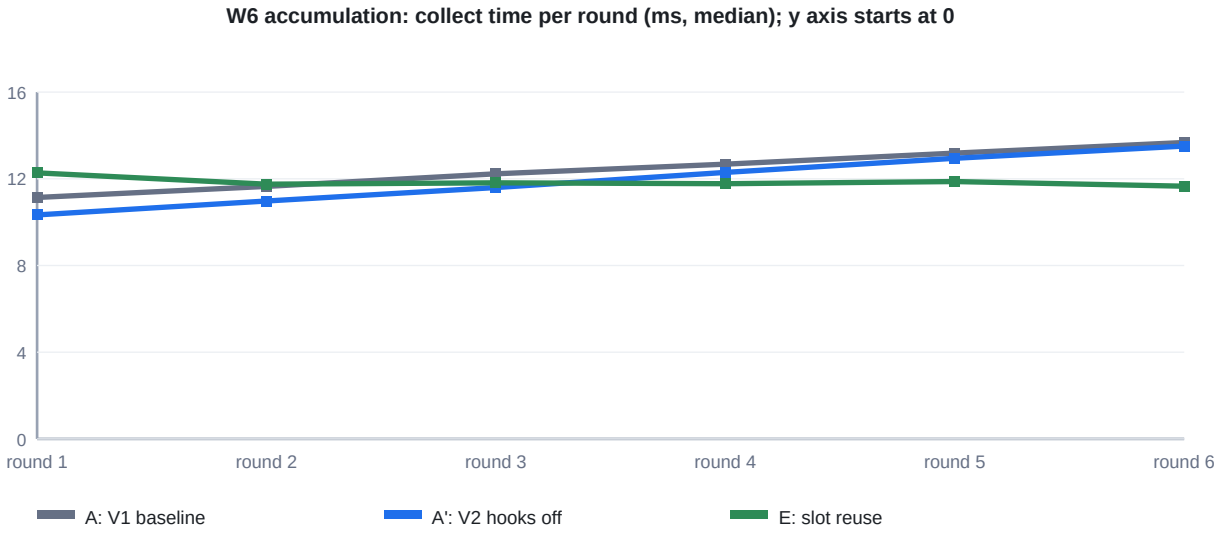


Figure 10 - Without slot reuse, collect time creeps up with every allocation ever made (V1 11.1 -> 13.7 ms, A' 10.3 -> 13.5 ms over six rounds, about +30%); with slot reuse it stays flat (11.7-12.3 ms). Audit finding D1, measured; smaller on this Mac than in the Linux audit, same shape.



Figure 11 - Per-event flushing (B=1) is 50-250x slower than not tracing; batching to 1,024 removes 83-94% of that overhead, and 8,192 adds little.

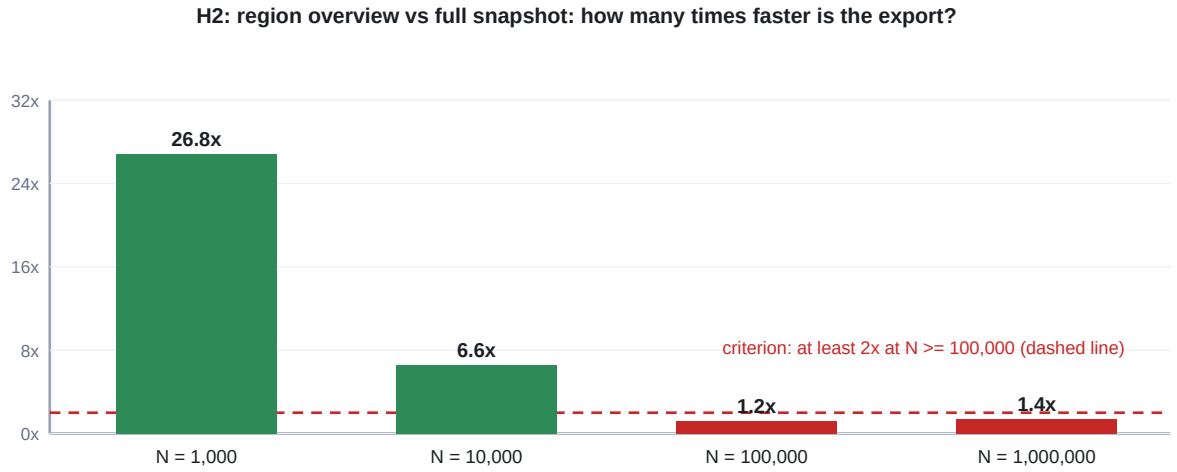


Figure 12 - The negative result: region overview vs full snapshot. With no locality in the graph the overview is nearly as large as the graph.

11.3 How to read the results

- **Safe to claim:** the refinements do not change what is freed; batching helps a lot; slot reuse fixes the scan-growth defect; regions are cheap unless every operation is a write; small overviews are much cheaper than snapshots.
- **Not proven:** that region overviews help for large heaps (the H2 graph had no locality); any Linux result; any speed comparison with HotSpot (never made).
- **Honest cost:** V2 with every feature off is still slower than V1 on the whole run (6-15%, 43% on W5) because setField now validates its target and objects are bigger. The collect-time criterion passes; the mutator cost is real and listed as an optimisation target.

HotSpot reference runs (J1-J5, 64 MiB heap, 10 runs each): behaviour only, never a speed comparison. Heap sizes are quantised to MiB by the GC log format.

Program	Collector	GCs	Pause total ms	Heap after last GC	Wall ms
J1	g1	101	96.4	1 MiB	146
J1	serial	101	67.1	1 MiB	117
J2	g1	40	68.8	1 MiB	115
J2	serial	40	50.7	1 MiB	98
J3	g1	20	36.5	1 MiB	80
J3	serial	20	27.3	1 MiB	73
J4	g1	3	5.1	1 MiB	47
J4	serial	3	3.8	1 MiB	46
J5	g1	11	23.0	1 MiB	67
J5	serial	11	15.8	2 MiB	59

12. Project history and size

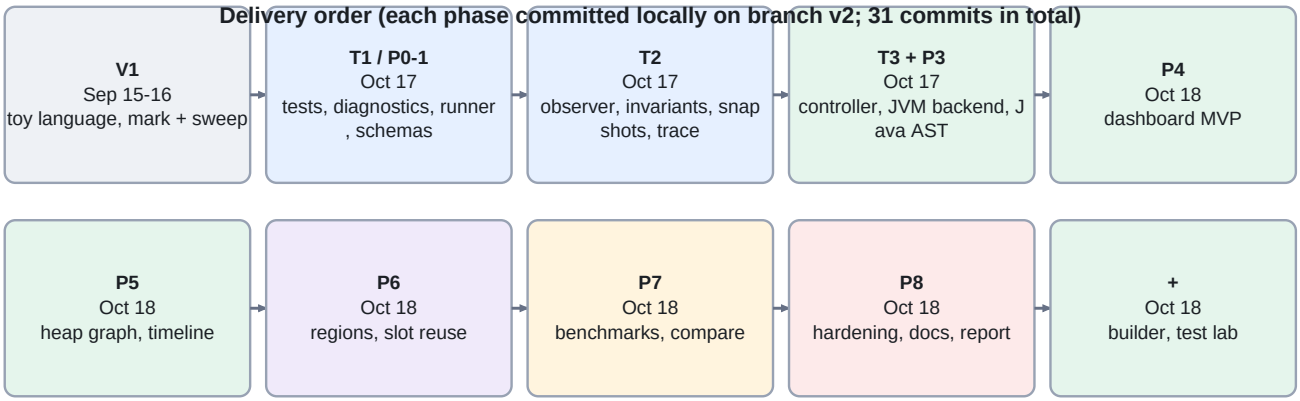


Figure 13 - Phases in delivery order.

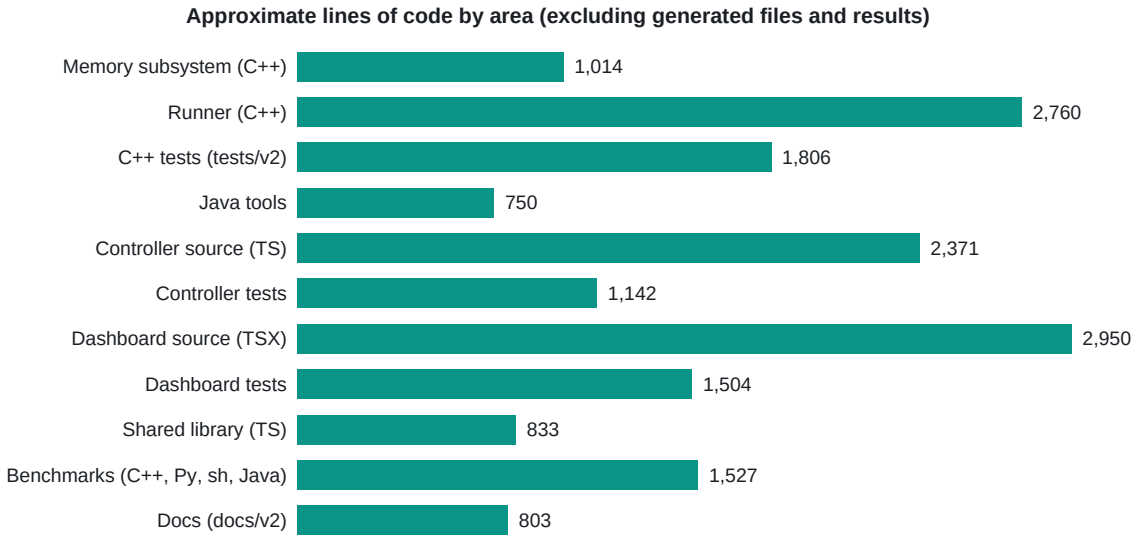


Figure 14 - V1 was about 1,150 lines of C++ in total (interpreter, runtime and collector); V2 adds roughly 15,000 lines of code, tests and documentation.

12.1 Feature checklist by phase

Phase	Delivered
T1 (P0-P1)	Baseline verified and tagged; V2 build scaffolding; always-on CHECK; correctness matrix and property test; GcDiagnostics; gcd_runner; schemas with examples
T2	GcObserver and HeapView; invariants I1-I5, I7; bounded snapshots; EventBuffer with file and memory sinks; trace schemas; replay test
T3 + P3	Java analyzer, launcher, build script; Fastify controller, adapters, GC-log parser, limits and cleanup; Java AST with spans, parse-only analyze route
P4	Dashboard MVP: toolbar, editor, AST, statistics with provenance, output, polling, static serving by the controller
P5	Trace and snapshot routes; heap graph with snapshot slider and 'Why alive?'; timeline; trace banners
P6	RegionTable and barrier, slot reuse and generations, stale-write hardening, runner flags, region route and panel, differential and region tests
P7	heap_bench harness, JVM workloads, orchestrator, analysis script, committed results, Compare and Benchmarks panels
P8	Security review fixes, setup/start scripts, README and docs, draft report
Extra	Object builder, CLI generator, test lab with predictions

13. Practical cheat sheet

13.1 Setup and start

```
scripts/setup.sh --check     # what is missing? (CMake >= 3.15, C++17, JDK 21, Node 22)
scripts/setup.sh             # builds runner, Java tools, web workspaces (about 30 s with a warm npm cache)
scripts/start.sh             # http://127.0.0.1:4177/   (finds JDK 21 and Node 22 even if defaults differ)

cd web && npm run dev        # controller with live reload (needs Node 22 and JDK 21 on PATH)
cd web && npm run dev:client # Vite dev server on :5173, proxied to the controller
```

13.2 Tests and tools

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug && cmake --build build -j && ctest --test-dir build
cmake -S . -B build-asan -DCMAKE_BUILD_TYPE=Debug -DGCD_SANITIZE=ON      # AddressSanitizer + UBSan
GCD_PROPERTY_SEEDS=10000 ./build/gc_property_tests                         # long seeded runs
cd web && npm run build && npm run lint && npm test && npm run smoke
cd web && npm run lab -- --fuzz 20 --seed 3      # lab against a running controller
node web/scripts/gen-toy.mjs --objects 100 --cycle-share 50 --seed 1 > p.gc ; build/gcd_runner --source p.gc --res
bench/run_bench.sh           # about an hour; --quick checks the pipeline in seconds
```

13.3 Things to remember

Topic	Detail
Ports and dirs	Controller 4177 (GCD_PORT); run directory <tmp>/gcd-runs-<port> (GCD_RUN_ROOT); lock file .controller.pid
Toolchains	JDK 21 required (health reports 'supported'); default JDK 25 or Node 25/26 on this machine will not do. setup/start scripts find the right ones.
After editing the dashboard	npm run build in web/ - the controller serves web/client/dist and re-indexes it on a miss
Where the full log is	docs/v2/PROGRESS.md (every phase, deviations, open issues); docs/v2/report.md (draft report)
Git	Branch v2, 31 local commits, nothing pushed. PLAN.md (the original plan, 1,359 lines) is untracked on purpose-undecided.

13.4 Troubleshooting

- **'dyld: Library not loaded: libsimdjson'** - the Homebrew default Node is broken; use Node 22 (/opt/homebrew/opt/node@22/bin) or scripts/start.sh.
- **Dashboard blank after a rebuild** - reload the page; the controller re-indexes new asset names on a miss.
- **409 BUSY** - another run (or the test lab) is active. **'Another controller (pid N) is already using ...'** - a controller already owns that port's run directory.
- **Java runs fail with 'JDK found: false'** - set JAVA_HOME to a JDK 21 or use scripts/start.sh.

14. Limitations and what is not verified

Item	Status
Linux	Never run on Linux: prlimit path untested; the 'clean Ubuntu VM in 15 minutes' acceptance was not checked (a macOS clean clone took about 30 s).
Benchmark conditions	One machine, battery power, no CPU pinning; batching grid at 0.25 scale; 30 samples from 3 processes.
H2	Failed on a graph with no locality; the favourable case was not measured.
Mutator overhead	V2 with features off costs 6-43% whole-run over V1 (setField validation, larger objects).
Dashboard	Stop button and automatic compile-error cursor jump only unit-tested; Cytoscape canvas checked by eye (tests mock it); no light-theme review; graphs near 2,000 objects not looked at.
Builder	Cannot read hand-written code back into settings; about 1,300 objects max (one variable per object, 64 KiB limit).
Report	docs/v2/report.md is a Markdown draft; some references (marked) need DOIs confirmed.
Security	Not a sandbox. See section 9.
Deferred by design	Region-based partial collection (H6), JFR UI, live streaming, multi-file Java, hardened sandbox, Windows, line/column in the toy language.

15. Glossary

Term	Meaning
Root	A variable (name) the interpreter keeps; objects reachable from roots are live.
Handle	Opaque value naming an object; here the slot index, or generation << 32 | slot with reuse.
Mark / sweep	Phase 1 marks everything reachable; phase 2 frees the unmarked and clears marks.
Slot	Index into the heap's object table.
Generation	A per-slot reuse counter (not an object age); makes stale handles detectable.
Region	Block of R consecutive slots used to summarise the heap; not a lifetime region and not collected separately.
Write barrier	Code run on every setField that keeps region edge counts exact.
Mark forest	First-discovery edges recorded during mark; each live object has exactly one discoverer; basis of 'Why alive?'.
Observer	Interface the runner implements to receive collection events and heap views.
Snapshot	Heap state at mark-complete (survivors and victims together) or program end; at most 2,000 objects.
Provenance	How a number was obtained: measured, estimated, reconstructed or unavailable.
Variant A, A', B, C, D, E	V1 baseline; V2 all off; regions; tracing; regions + tracing; slot reuse.
Differential test	Run the same random program on several variants and require identical freed sets.
Digest	Hash of every collection's (freed, live) pair, compared across variants in the benchmarks.
Process group	Set of processes killed together; each run leads its own.